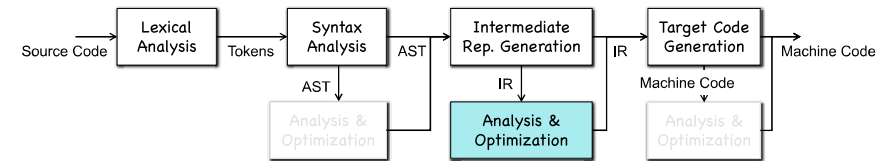


## Chapter 12-2 Pointer Alias Analysis

## Pointer Analysis



### • Why pointer analysis?

`*p = a; ...; b = *q; // does b data-depend on a?`

Pointer analysis is the most fundamental program analysis!

- Flow-sensitivity
- Context-sensitivity
- Path-sensitivity
- Field-sensitivity

5

## Why Pointer Alias Analysis?

- **Aliases:** two expressions that denote the same memory location

- Aliases are introduced by

```

int x,y;
int *p = &x;
int *q = &y;
int *r = p;
int **s = &q;
  
```

- references/pointers
- function call
- array indexing
- C unions
- ...

8

## Why Pointer Alias Analysis?

- Improving the precision of **compiler optimizations** that require knowing what is modified or referenced

```

x = 3;
*p = 4;
y = x; // can constant 3 propagate here
  
```

Constant Propagation

```

t = a + b;
*p = t;
y = a + b; // is a + b available?
  
```

Available Expression

- Error detection

```

x.lock();
...
y.unlock(); // same object as x?
  
```

12

## May/Must Pointer Alias Analysis

- **May- (Must-)** analysis checks if two expressions **may (must)** denote the same memory location.

- May analysis – Must-not analysis

- If a *may-analysis* says 'no', it means two expressions must not denote the same memory location.
- Useful in compiler optimization.

```

x = 3;
*p = 4;
y = x; // can constant 3 propagate here
  
```

Constant Propagation

```

t = a + b;
*p = t;
y = a + b; // a + b is available?
  
```

Available Expression

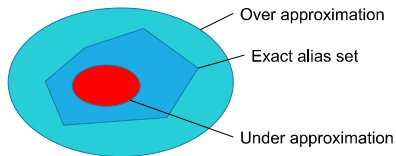
## PART I: Pointer Alias Analysis

13

16

## May/Must Pointer Alias Analysis

- Sound/Complete or Over-approximation/Under-approximation
- May analysis provides a sound (or over-approximated) alias set



The simplest sound alias analysis is to return true for all may-alias queries

- Tradeoff between soundness and precision (i.e., completeness)

22

## Flow-Sensitive Pointer Analysis

- Flow-sensitive pointer analysis computes **for each program point** what memory locations a pointer expression may refer to
- Flow-sensitive pointer analysis is **(traditionally) too expensive** to perform for whole program
- There have been a few studies for flow-sensitive or even path-sensitive analysis

[1] **Falcon: A Fused Approach to Path-Sensitive Data Dependence Analysis**  
Peisen Yao, Jinguo Zhou, Xiao Xiao, Qingkai Shi, Rongxin Wu, and Charles Zhang, PLDI 2024.

[2] **Value-Flow-Based Demand-Driven Pointer Analysis for C and C++**  
Yulei Sui and Jingling Xue, IEEE Transactions on Software Engineering 2018.

26

## A Small Pointer Language

- $y = \&x$  (address-taken)
- $y = x$  (assignment)
- $*y = x$  (store statement)
- $y = *x$  (load statement)
- **pts(x)** --- the points-to set of x
  - $\text{pts}(x) = \{y, z\}$  --- x may point to either y or z

29

## Flow-Sensitive Pointer Analysis

- Flow-sensitive pointer analysis computes **for each program point** what memory locations a pointer expression may refer to
- Flow-sensitive pointer analysis is **(traditionally) too expensive** to perform for whole program
- There have been a few studies for flow-sensitive or even path-sensitive analysis

25

## Flow-Insensitive Pointer Analysis

- Flow-insensitive pointer analyses for whole program analyses

**Andersen's Algorithm**  
Presented by Andersen in 1994  
Nearly cubic complexity

**Steensgaard's Algorithm**  
Published in POPL 1996  
Almost linear,  $O(n\alpha(n))$

27

## Andersen's Algorithm

Inclusion-based pointer analysis

| Statement | Constraint                                                           | Shorthand                                |
|-----------|----------------------------------------------------------------------|------------------------------------------|
| $y = \&x$ | $\text{pts}(y) \supseteq \{x\}$                                      |                                          |
| $y = x$   | $\text{pts}(y) \supseteq \text{pts}(x)$                              |                                          |
| $*y = x$  | $\forall v \in \text{pts}(y): \text{pts}(v) \supseteq \text{pts}(x)$ | $\text{pts}(*y) \supseteq \text{pts}(x)$ |
| $y = *x$  | $\forall v \in \text{pts}(x): \text{pts}(y) \supseteq \text{pts}(v)$ | $\text{pts}(y) \supseteq \text{pts}(*x)$ |

```
p = &a;
q = &b;
*p = q;
r = &c;
s = p;
t = *p;
*s = r;
```

?

```
p = &a;
q = p;
p = &b;
r = p;

pts(p) ⊇ { a };
pts(q) ⊇ pts(p);
pts(p) ⊇ { b };
pts(r) ⊇ pts(p);

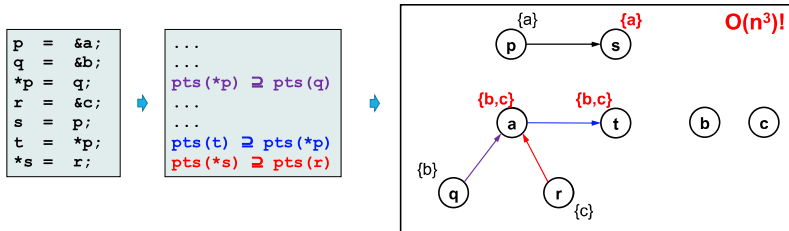
pts(p) = { a, b };
pts(q) = { a, b };
pts(r) = { a, b };
pts(a) = pts(b) = {};
```

Over-approximation!  
Sound result!

44

## Andersen's Alg. as Graph Closure

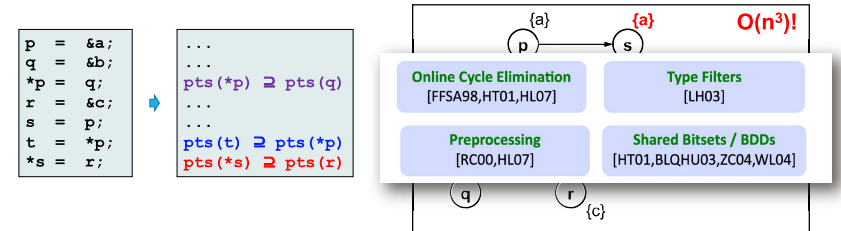
- Each graph node  $x$  denotes the points-to set  $pts(x)$
- Solving the set constraints via a dynamic transitive closure



53

## Andersen's Alg. as Graph Closure

- Each graph node  $x$  denotes the points-to set  $pts(x)$
- Solving the set constraints via a dynamic transitive closure



54

## Steensgaard's Algorithm

Inclusion-based pointer analysis (Andersen's Algorithm)

| Statement | Constraint                                      | Shorthand                  |
|-----------|-------------------------------------------------|----------------------------|
| $y = \&x$ | $pts(y) \supseteq \{x\}$                        |                            |
| $y = x$   | $pts(y) \supseteq pts(x)$                       |                            |
| $*y = x$  | $\forall v \in pts(y): pts(v) \supseteq pts(x)$ | $pts(*y) \supseteq pts(x)$ |
| $y = *x$  | $\forall v \in pts(x): pts(y) \supseteq pts(v)$ | $pts(y) \supseteq pts(*x)$ |

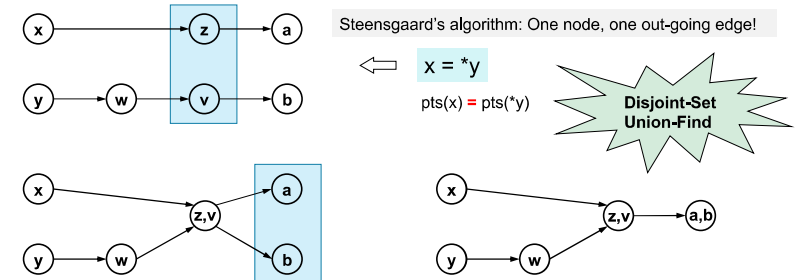
Unification-based pointer analysis (Steensgaard's Algorithm)

| Statement | Constraint                              | Shorthand          |
|-----------|-----------------------------------------|--------------------|
| $y = \&x$ | $pts(y) \supseteq \{x\}$                |                    |
| $y = x$   | $pts(y) = pts(x)$                       |                    |
| $*y = x$  | $\forall v \in pts(y): pts(v) = pts(x)$ | $pts(*y) = pts(x)$ |
| $y = *x$  | $\forall v \in pts(x): pts(y) = pts(v)$ | $pts(y) = pts(*x)$ |

57

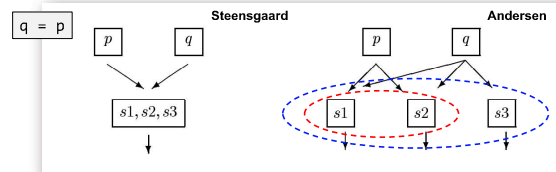
## Steensgaard's Algorithm

- Points-to graph:  $x \rightarrow y$  means  $y \in pts(x)$ .



67

## Steensgaard's Algorithm



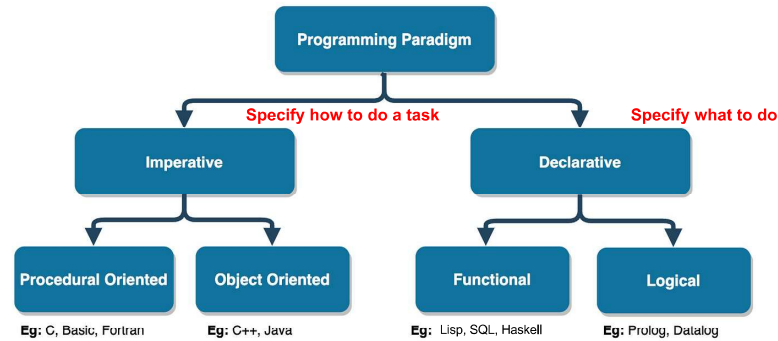
| Steensgaard                      | Andersen        |
|----------------------------------|-----------------|
| Sound                            | Sound           |
| Almost linear (more efficient)   | Nearly cubic    |
| Unification-based (less precise) | Inclusion-based |

70

## PART IV: Datalog-Based DFA

71

## Recap: Programming Paradigm



74

## Logic Programming

### • Example:

- Nanjing is rainy
  - Beijing is rainy
  - Beijing is cold
  - If a city is both rainy and cold, then it is snowy
- } **Facts**
- } **Rules**
- **Query:** which city is snowy
  - Search for solutions based on facts and rules

80

## What is Datalog?

- Datalog is a subset of Prolog
  - All Datalog programs terminate
  - Ordering of rules does not matter
  - Not Turing complete
- Souffle Datalog engine --- <https://souffle-lang.github.io/>
  - .decl rainy (c:symbol)
  - .decl cold(c:symbol)
  - .decl snowy(c:symbol)
  - .output snowy
  - rainy("Nanjing")
  - rainy("Beijing")
  - cold("Beijing")
  - snowy(c) :- rainy(c), cold(c)

84

## Logic Programming

- Logic programming
  - In a broad sense: the use of mathematical logic for programming
- Prolog (1972)
  - Use logical rules to specify how mathematical relations are computed
  - A prolog program is a database of logical rules

76

## Logic Programming

- Expert systems
- Natural language processing
- Theorem provers
- Reasoning about safety a security
- **Program analysis/Compiler optimization**

81

## Predicate/Atom

- Predicates are N-ary relations
  - predicate(x, y, z)
- Examples
  - rainy("Nanjing")
  - rainy("Beijing")
  - cold("Beijing")

85

## Predicate/Atom

- Predicates are N-ary relations
  - $\text{predicate}(x, y, z)$
- Examples
  - $\text{rainy}(\text{"Nanjing"})$
  - $\text{rainy}(\text{"Beijing"})$
  - $\text{cold}(\text{"Beijing"})$
  - $\text{brother}(x, y) \dashv\vdash x \text{ is } y\text{'s brother}$
  - $\text{speaks}(x, a) \dashv\vdash x \text{ speaks language } a$

86

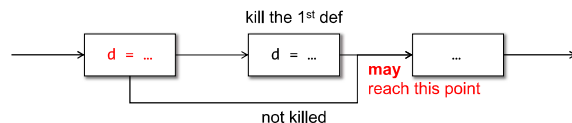
## Datalog Programming Model

- All rules hold for any instantiation of its variables
  - $\text{snowy}(c) \text{ :- } \text{rainy}(c), \text{cold}(c)$  for all cities like Nanjing, Beijing, etc....
- Anything not declared is not true
- Ordering of rules does not matter for results (Prolog vs. Datalog)
- Rules may be recursive
  - $\text{reachable}(a, b) \text{ :- } \text{edge}(a, b);$
  - $\text{reachable}(a, c) \text{ :- } \text{edge}(a, b), \text{reachable}(b, c)$

92

## Recap: Reaching Definition

- A definition  $d$  **may** reach a program point, if **there is a path from  $d$  to the program point** such that  $d$  **is not killed along the path**.



97

## Datalog Programming Model

- A Datalog program is a database of Horn clauses
  - $h$  is true **if** the assumptions  $l_1 \ l_2 \ \dots \ l_n$  are simultaneously true
  - **if** --- a sufficient but not necessary condition

$$h \text{ :- } l_1 \ l_2 \ \dots \ l_n$$

- **Example:**
  - $\text{rainy}(\text{"Nanjing"})$
  - $\text{snowy}(c) \text{ :- } \text{rainy}(c), \text{cold}(c)$

88

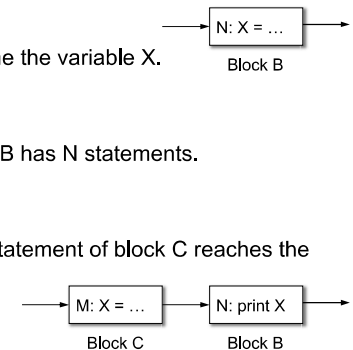
## Datalog Programming Model

- Negation is allowed in assumptions
  - $\text{more\_than\_one\_hop}(a, b) \text{ :- } \text{reachable}(a, b), \neg \text{edge}(a, b)$
- All rules must be well-formed. Example of bad rules:
  - $\text{more\_than\_one\_hop}(a, b) \text{ :- } \neg \text{edge}(a, b)$
  - **Bad:**  $a, b$  on the left do not appear in the positive predicate on the right
  - **Goal:** to ensure termination

95

## Reaching Definitions by Datalog

- $\text{def}(B, N, X)$ 
  - the  $N$ th statement in Block  $B$  may define the variable  $X$ .
- $\text{succ}(B, N, C)$ 
  - block  $C$  is a successor of block  $B$ , and  $B$  has  $N$  statements.
- $\text{rd}(B, N, C, M, X)$ 
  - the definition of variable  $X$  at the  $M$ th statement of block  $C$  reaches the  $N$ th statement in  $B$ .



## Reaching Definitions by Datalog

- $rd(B, N, B, N, X) :- \text{def}(B, N, X)$
- $rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), \text{def}(B, N, Y), X \neq Y$
- $rd(B, 0, C, M, X) :- rd(D, N, C, M, X), \text{succ}(D, N, B)$

101

## Reaching Definitions by Datalog

- $rd(B, N, B, N, X) :- \text{def}(B, N, X)$
  - $rd(B, N, C, M, X) :- rd(B, N-1, C, M, X), \text{def}(B, N, Y), X \neq Y$
  - $rd(B, 0, C, M, X) :- rd(D, N, C, M, X), \text{succ}(D, N, B)$
- |                           |                              |                              |
|---------------------------|------------------------------|------------------------------|
| • $\text{def}(B_1, 1, i)$ | • $\text{def}(B_2, 2, j)$    | • $\text{succ}(B_2, 2, B_3)$ |
| • $\text{def}(B_1, 2, j)$ | • $\text{def}(B_3, 1, a)$    | • $\text{succ}(B_2, 2, B_4)$ |
| • $\text{def}(B_1, 3, a)$ | • $\text{def}(B_4, 1, i)$    | • $\text{succ}(B_3, 1, B_4)$ |
| • $\text{def}(B_2, 1, i)$ | • $\text{succ}(B_1, 3, B_2)$ | • $\text{succ}(B_4, 1, B_2)$ |

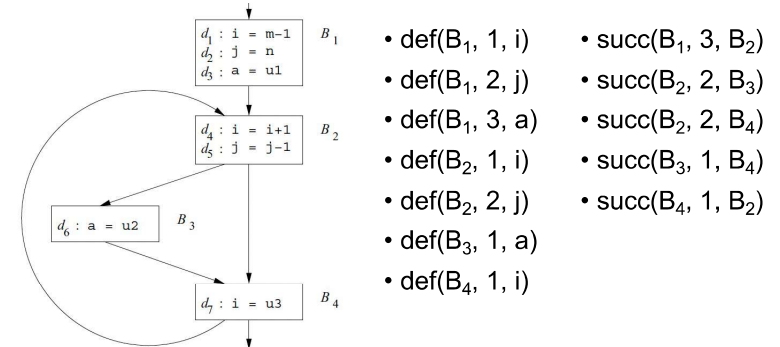
108

## Pointer Analysis by Datalog

- Four kinds of Java statements and ignore procedural call
  - **Object creation.**  $h: T \ v = \text{new } T();$
  - **Copy.**  $v = w;$
  - **Field store.**  $v.f = w;$
  - **Field load.**  $v = w.f;$
- $\text{pts}(V, H)$  variable V can point to a heap object H
  - $\text{hpts}(H, F, G)$  field F of heap object H can point to heap obj G

113

## Reaching Definitions by Datalog



- $\text{def}(B_1, 1, i)$
- $\text{def}(B_1, 2, j)$
- $\text{def}(B_1, 3, a)$
- $\text{def}(B_2, 1, i)$
- $\text{def}(B_2, 2, j)$
- $\text{def}(B_3, 1, a)$
- $\text{def}(B_4, 1, i)$
- $\text{succ}(B_1, 3, B_2)$
- $\text{succ}(B_2, 2, B_3)$
- $\text{succ}(B_2, 2, B_4)$
- $\text{succ}(B_3, 1, B_4)$
- $\text{succ}(B_4, 1, B_2)$

107

## Reaching Definitions by Datalog

- $rd(B, N, B, N, X) :- \text{def}(B, N, X)$

We just define facts and rules.  
Analysis is automatically done by Datalog engines!

- $rd(B_1, 1, i)$
- $rd(B_1, 2, j)$
- $rd(B_1, 3, a)$
- $rd(B_2, 1, i)$
- $rd(B_2, 2, j)$
- $rd(B_3, 1, a)$
- $rd(B_4, 1, i)$
- $rd(B_4, 2, B_3)$
- $\text{def}(B_1, 1, i)$
- $\text{def}(B_1, 2, j)$
- $\text{def}(B_1, 3, a)$
- $\text{def}(B_2, 1, i)$
- $\text{def}(B_2, 2, j)$
- $\text{def}(B_3, 1, a)$
- $\text{def}(B_4, 1, i)$
- $\text{succ}(B_2, 2, B_3)$
- $\text{succ}(B_2, 2, B_4)$
- $\text{succ}(B_3, 1, B_4)$
- $\text{succ}(B_4, 1, B_2)$

**Query Example:**  $rd(B_4, 1, B_1, 1, i)$

110

## Pointer Analysis by Datalog

- 1)  $\text{pts}(V, H) :- "H : T \ V = \text{new } T"$
- 2)  $\text{pts}(V, H) :- "V = W" \ \& \ \text{pts}(W, H)$
- 3)  $\text{hpts}(H, F, G) :- "V.F = W" \ \& \ \text{pts}(W, G) \ \& \ \text{pts}(V, H)$
- 4)  $\text{pts}(V, H) :- "V = W.F" \ \& \ \text{pts}(W, G) \ \& \ \text{hpts}(G, F, H)$

114

# Interprocedural Pointer Analysis

- Dealing with method invocation in Java programs
- $x = y.n(z)$
- actual**(S, I, V): V is the I<sup>th</sup> argument in call site S
- formal**(M, I, V): V is the I<sup>th</sup> parameter in method M

|                                                                                                                                                             |                                                                                                            |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------|
| 1) $invokes(S, N) \quad :- \quad "S : V.N(...)"$                                                                                                            | 2) $pts(V, H) \quad :- \quad invokes(S, M) \ \& \ formal(M, I, V) \ \& \ actual(S, I, W) \ \& \ pts(W, H)$ |
| Just follow the rule for assignment $V = W$                                                                                                                 |                                                                                                            |
| It is <b>context-insensitive</b> as we always do the same thing when calling a function, i.e., do not distinguish different call sites of the same function |                                                                                                            |

124

# Context-Sensitive Pointer Analysis

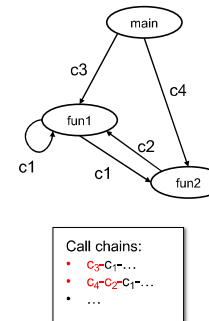
- invokes**(S, **C**, M, **D**)
- call site S in context C, invokes the method M of context D
- pts**(V, **C**, H)
- V points to H in context C
- hpts**(H, F, G)
- same as before, field F of H points to G

call chains of length  $\leq$  a predefined value

131

## PART V: Object Sensitivity

# Context-Sensitive Pointer Analysis



- Assume we already have the call graph.
- Calling context:
  - call chains, paths in the graph
- Context-sensitive analysis:
  - analyze functions in different calling context
- Difficulty:** Infinite # calling contexts
- Solution:** restrict the length of call chains

127

# Context-Sensitive Pointer Analysis

|   | Context-Insensitive                                                                         | Context-Sensitive                                                                                                                           |
|---|---------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | $pts(V, H) \quad :- \quad "H : T \ V = new \ T()"$                                          | $pts(V, \mathbf{C}, H) \quad :- \quad "H : T \ V = new \ T()", \ \mathbf{invoke}(H, \mathbf{C}, \_, \_)"$                                   |
| 2 | $pts(V, H) \quad :- \quad "V = W", \ pts(W, H)$                                             | $pts(V, \mathbf{C}, H) \quad :- \quad "V = W", \ pts(W, \mathbf{C}, H)$                                                                     |
| 3 | $hpts(H, F, G) \quad :- \quad "V, F = W", \ pts(W, G), \ pts(V, H)$                         | $hpts(H, F, G) \quad :- \quad "V, F = W", \ pts(W, \mathbf{C}, G), \ pts(V, \mathbf{C}, H)$                                                 |
| 4 | $pts(V, H) \quad :- \quad "V = W, F", \ pts(W, G), \ hpts(G, F, H)$                         | $pts(V, \mathbf{C}, H) \quad :- \quad "V = W, F", \ pts(W, \mathbf{C}, G), \ hpts(G, F, H)$                                                 |
| 5 | $pts(V, H) \quad :- \quad invokes(S, M), \ formal(M, I, V), \ actual(S, I, W), \ pts(W, H)$ | $pts(V, \mathbf{D}, H) \quad :- \quad invokes(S, \mathbf{C}, M, \mathbf{D}), \ formal(M, I, V), \ actual(S, I, W), \ pts(W, \mathbf{C}, H)$ |

136

# Object Sensitivity

```

class List {
    int x;
    void foo() {
        print(x);
    }
}

class A {
    void bar(List a) {
        a.foo();
        a.foo();
    }
}

```

A new form of context-sensitivity, especially for OO programs. It does not distinguish contexts based on call sites.

An object-sensitive analysis will not separately analyze  $c_2c_1$  and  $c_3c_1$ , because they use the same object a.

137

141